이 파트에서는 지금까지 학습한 모든 기술을 통합하여 **실전 웹 애플리케이션**을 개발합니다.

Flask 프레임워크로 웹 개발의 기초를 익힌 후, AI API 연동, 엑셀 데이터 처리, 검색 기능, 챗봇 인터페이스를 갖춘 비즈니스 앱을 만들고 배포까지 진행합니다.

학습 목표

- Flask를 사용하여 웹 애플리케이션의 기본 구조를 만들 수 있습니다.
- AI API, 엑셀 연동, 검색 기능을 통합한 비즈니스 앱을 개발할 수 있습니다.
- ngrok으로 로컬 앱을 외부에 공개하고 클라우드에 배포할 수 있습니다.

이 파트의 구성

챕터	제목	핵심 내용
Chapter 21	웹 애플리케이션 기초	Flask, 라우팅, 템플릿, 폼 처리
Chapter 22	AI 기반 웹 애플리케이션 개발	비즈니스 로직, 엑셀 연동, 챗봇 UI
Chapter 23	배포와 운영	ngrok, 클라우드 배포, 보안 관리

이번 챕터에서는 AI 모델을 실제 서비스로 배포하기 위해 반드시 알아야 하는 웹 애플리케이션의 기초를 다룹니다. 웹의 동작 원리부터 시작해, Python 기반의 경량 웹 프레임워크인 Flask를 사용해 간단한 웹앱을 직접 만들어 봅니다.

> *이 챕터를 마치면 Flask로 라우팅, 템플릿 렌더링, 폼 데이터 처리, 정적 파일 관리까지 갖춘 기본 웹앱을 구축할 수 있습니다.*

21.1 웹의 기본 구조

클라이언트-서버 모델

웹은 **클라이언트-서버(Client-Server)** 모델을 기반으로 동작합니다. 사용자의 브라우저가 **클라이언트(Client)**가 되어 서버에 요청을 보내면, **서버(Server)**는 그 요청을 처리하고 응답을 반환합니다.

클라이언트-서버 HTTP 통신



프론트엔드(HTML·CSS·JS)는 사용자가 보는 화면을, 백엔드(Python·DB·AI)는 데이터 처리와 로직을 담당

HTTP 프로토콜

HTTP(HyperText Transfer Protocol)는 클라이언트와 서버가 데이터를 주고받는 규칙(프로토콜)입니다. 현재는 보안이 강화된 **HTTPS(HTTP Secure)**가 표준으로 사용됩니다.

HTTP 통신에서 자주 사용되는 **메서드(Method)**는 다음과 같습니다.

메서드	역할	사용 예시
GET	데이터 조회	웹 페이지 열기, 검색
POST	데이터 전송/생성	로그인, 폼 제출
PUT	데이터 수정(전체)	회원 정보 수정
DELETE	데이터 삭제	게시글 삭제

HTTP 응답에는 결과를 나타내는 **상태 코드(Status Code)**가 포함됩니다.

코드	의미
200	OK (성공)
301/302	Redirect (리다이렉트)
404	Not Found (페이지 없음)
500	Internal Server Error (서버 오류)

프론트엔드와 백엔드

웹 개발은 역할에 따라 두 영역으로 나뉩니다.

구분	프론트엔드(Frontend)	백엔드(Backend)
역할	사용자가 보는 화면 담당	데이터 처리 및 로직 담당
위치	브라우저(클라이언트)	서버
주요 기술	HTML, CSS, JavaScript	Python, Flask, DB

AI 모델은 대부분 **백엔드** 영역에서 실행되며, 프론트엔드는 사용자 입력을 받아 백엔드로 전달하고 결과를 화면에 표시하는 역할을 합니다.

21.2 Flask 프레임워크 소개 및 설치

Flask란?

Flask는 Python으로 웹 애플리케이션을 만들 수 있는 경량 **마이크로 웹 프레임워크(Micro Web Framework)**입니다. 핵심 기능만 제공하고 나머지는 필요에 따라 확장하는 방식으로, 빠른 프로토타이핑과 AI 모델 서빙에 매우 적합합니다.

> *Django와 달리 Flask는 최소한의 구성으로 시작할 수 있어, AI/ML 실험을 빠르게 웹으로 전환할 때 특히 유용합니다.*

Flask vs Django 비교

항목	Flask	Django
철학	마이크로프레임워크, 자유도 높음	풀스택, 배터리 포함
학습 곡선	낮음	상대적으로 높음
적합한 규모	소-중형, API 서버, AI 서빙	중-대형 웹 서비스
ORM 기본 제공	없음 (Flask-SQLAlchemy 확장)	있음

설치

가상환경을 활성화한 상태에서 pip으로 설치합니다.

```
pip install flask
```

설치 확인은 다음 명령어로 합니다.

```
python -c "import flask; print(flask.__version__)"
```

최소 Flask 앱

Flask 앱의 가장 기본적인 형태는 다음과 같습니다.

```
from flask import Flask

app = Flask(__name__)   # Flask 앱 인스턴스 생성

@app.route("/")         # 루트 URL에 대한 라우트 등록
def index():
    return "Hello, Flask!"

if __name__ == "__main__":
    app.run(debug=True) # 개발 서버 실행 (debug 모드)
```

`app.run(debug=True)`를 실행하면 기본적으로 `http://127.0.0.1:5000`에서 개발 서버가 시작됩니다.

> ***debug=True**는 코드 변경 시 서버가 자동으로 재시작되고 오류 정보를 브라우저에 표시해 주므로, 개발 중에는 편리하지만 운영 환경에서는 반드시 `False` 로 설정해야 합니다.*

21.3 라우팅(Routing)과 URL 매핑

라우팅이란?

라우팅(Routing)은 특정 URL로 들어온 요청을 어떤 함수(뷰 함수)가 처리할지 연결하는 것을 말합니다. Flask에서는 `@app.route()` 데코레이터로 라우트를 등록합니다.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
    return "홈 페이지입니다."

@app.route("/about")
def about():
    return "소개 페이지입니다."

@app.route("/hello/<name>")   # URL 변수 사용
def hello(name):
    return f"안녕하세요, {name}님!"
```

`<name>` 처럼 꺾쇠 괄호를 사용하면 URL의 일부를 **변수(Variable Rule)**로 받을 수 있습니다.

HTTP 메서드 지정

기본적으로 `@app.route()` 는 **GET** 메서드만 허용합니다. 여러 메서드를 허용하려면 `methods` 인자를 지정합니다.

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/submit", methods=["GET", "POST"])
def submit():
    if request.method == "POST":
        return "POST 요청을 받았습니다."
    return "GET 요청을 받았습니다."
```

URL 빌드: url_for()

하드코딩된 URL 대신 `url_for()` 함수를 사용하면 함수 이름으로 URL을 동적으로 생성할 수 있어 유지보수가 편리합니다.

```
from flask import Flask, url_for

app = Flask(__name__)

@app.route("/user/<username>")
def profile(username):
    return f"{username}의 프로필 페이지"

with app.test_request_context():
    # '/user/alice' 를 반환
    print(url_for("profile", username="alice"))
```

21.4 템플릿 엔진 (Jinja2)과 HTML 기초

템플릿 엔진이란?

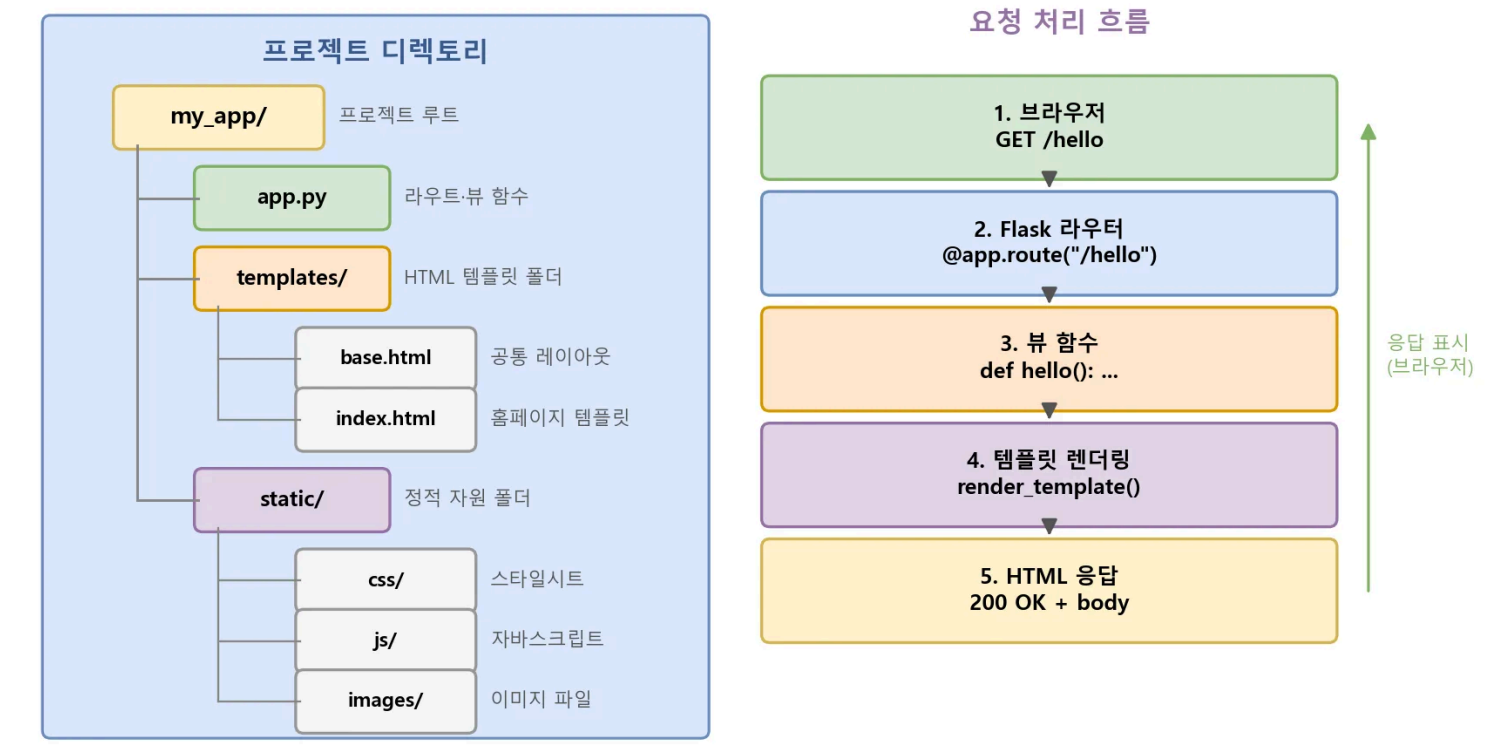
백엔드에서 동적으로 생성된 데이터를 HTML에 삽입하려면 **템플릿 엔진(Template Engine)**이 필요합니다. Flask는 **Jinja2**를 기본 템플릿 엔진으로 사용합니다.

Jinja2의 핵심 문법은 다음과 같습니다.

구문	용도	예시
`{{ }}`	변수 출력	`{{ username }}`
`{% %}`	제어문 (if, for)	`{% if user %}`
`{# #}`	주석	`{# 이건 주석 #}`

디렉토리 구조

Flask는 기본적으로 `templates/` 폴더에서 HTML 파일을 찾습니다.



기본 템플릿 작성

templates/base.html (레이아웃 기반 템플릿)

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}My App{% endblock %}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
    <header>
        <h1>My Flask App</h1>
        <nav>
            <a href="{{ url_for('index') }}">홈</a> |
            <a href="{{ url_for('about') }}">소개</a>
        </nav>
    </header>
    <main>
        {% block content %}{% endblock %}
    </main>
</body>
</html>
```

templates/index.html (자식 템플릿)

```
{% extends "base.html" %}

{% block title %}홈 - My App{% endblock %}

{% block content %}
  <h2>환영합니다!</h2>
  <p>안녕하세요, <strong>{{ username }}</strong>님.</p>

  <h3>메뉴 목록</h3>
  <ul>
    {% for item in menu_list %}
      <li>{{ item }}</li>
    {% endfor %}
  </ul>
{% endblock %}
```

Python에서 템플릿 렌더링

`render_template()` 함수로 HTML 파일을 렌더링하고 변수를 전달합니다.

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def index():
    menu = ["스테이크", "파스타", "샐러드"]
    return render_template("index.html", username="홍길동", menu_list=menu)
```

21.5 폼(Form) 데이터 처리

HTML 폼과 Flask

사용자로부터 입력을 받는 가장 기본적인 방법은 **HTML 폼(Form)**입니다. Flask에서는 `request` 객체를 통해 폼 데이터에 접근합니다.

templates/form.html

```
{% extends "base.html" %}
{% block title %}폼 예제{% endblock %}

{% block content %}
<h2>메시지 입력</h2>
<form action="/result" method="POST">
    <label for="message">메시지:</label>
    <input type="text" id="message" name="message" placeholder="내용을 입력하세요">
    <button type="submit">전송</button>
</form>
{% endblock %}
```

app.py

```python
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/form")
def form():
    return render_template("form.html")

@app.route("/result", methods=["POST"])
def result():
    message = request.form.get("message", "")   # POST 데이터에서 'message' 키 추출
    return render_template("result.html", user_message=message)
```

templates/result.html

```python
{% extends "base.html" %}
{% block title %}결과{% endblock %}

{% block content %}
<h2>입력하신 메시지</h2>
<p>{{ user_message }}</p>
<a href="/form">돌아가기</a>
{% endblock %}
```

GET vs POST 데이터 접근

데이터 종류	접근 방법	예시
POST 폼 데이터	`request.form.get("key")`	로그인 폼
GET 쿼리 파라미터	`request.args.get("key")`	검색어 (?q=flask)
JSON 데이터	`request.get_json()`	API 요청

21.6 정적 파일 관리 (CSS, JavaScript)

정적 파일 디렉토리

Flask는 `static/` 폴더에서 CSS, JavaScript, 이미지 등 정적 파일을 제공합니다. `url_for('static', filename='...')` 함수로 경로를 생성하는 것을 권장합니다.

```
static/
├── css/
│   └── style.css
├── js/
│   └── main.js
└── images/
    └── logo.png
```

CSS 파일 적용

static/css/style.css

```css
/* 기본 리셋 및 공통 스타일 */
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: 'Nanum Gothic', sans-serif;
    background-color: #f5f5f5;
    color: #333;
    line-height: 1.6;
}

header {
    background-color: #2c3e50;
    color: white;
    padding: 1rem 2rem;
}

header h1 {
    font-size: 1.5rem;
}

header nav a {
    color: #ecf0f1;
    text-decoration: none;
    margin-left: 1rem;
}

main {
    max-width: 800px;
    margin: 2rem auto;
    padding: 0 1rem;
}

button, input[type="submit"] {
    background-color: #3498db;
    color: white;
    border: none;
    padding: 0.5rem 1.2rem;
    border-radius: 4px;
```

```
    cursor: pointer;
}

button:hover {
    background-color: #2980b9;
}
```

HTML 템플릿에서 링크 방법은 다음과 같습니다.

```
<link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
```

JavaScript 파일 적용

static/js/main.js

```
// 페이지 로드 후 실행
document.addEventListener("DOMContentLoaded", function () {
    console.log("Flask 앱이 로드되었습니다.");
});
```

HTML 템플릿 `</body>` 태그 바로 위에 삽입합니다.

```
<script src="{{ url_for('static', filename='js/main.js') }}"></script>
```

[실습] Flask로 "Hello AI" 웹앱 만들기

지금까지 배운 내용을 종합해 사용자가 이름을 입력하면 AI처럼 인사하는 간단한 웹앱을 만들어 봅니다.

프로젝트 구조

```
hello_ai/
├── app.py
├── templates/
│   ├── base.html
│   ├── index.html
│   └── greet.html
└── static/
    └── css/
        └── style.css
```

1단계: app.py 작성

```
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

@app.route("/", methods=["GET"])
def index():
    """메인 페이지: 이름 입력 폼을 표시합니다."""
    return render_template("index.html")

@app.route("/greet", methods=["POST"])
def greet():
    """폼 데이터를 받아 인사 메시지를 생성합니다."""
    name = request.form.get("name", "").strip()
    if not name:
        return redirect(url_for("index"))   # 이름이 없으면 홈으로 리다이렉트
    message = f"안녕하세요, {name}님! 저는 AI 어시스턴트입니다. 무엇을 도와드릴까요?"
    return render_template("greet.html", name=name, message=message)

if __name__ == "__main__":
    app.run(debug=True)
```

2단계: 템플릿 작성

templates/base.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Hello AI{% endblock %}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
    <header>
        <h1>🤖 Hello AI</h1>
        <nav><a href="{{ url_for('index') }}">홈으로</a></nav>
    </header>
    <main>
        {% block content %}{% endblock %}
    </main>
</body>
</html>
```

templates/index.html

```
{% extends "base.html" %}
{% block title %}Hello AI - 시작{% endblock %}

{% block content %}
<div class="card">
    <h2>AI에게 인사해 보세요</h2>
    <form action="{{ url_for('greet') }}" method="POST">
        <label for="name">이름을 입력하세요:</label>
        <input type="text" id="name" name="name" placeholder="홍길동" required>
        <button type="submit">인사하기</button>
    </form>
</div>
{% endblock %}
```

templates/greet.html

```
{% extends "base.html" %}
{% block title %}Hello AI - 인사{% endblock %}

{% block content %}
<div class="card">
    <h2>AI의 응답</h2>
    <p class="message">{{ message }}</p>
    <a href="{{ url_for('index') }}">
        <button>다시 인사하기</button>
    </a>
</div>
{% endblock %}
```

3단계: CSS 작성

static/css/style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
    font-family: 'Segoe UI', sans-serif;
    background: #eef2f7;
    color: #333;
}

header {
    background: #2c3e50;
    color: white;
    padding: 1rem 2rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

header nav a { color: #bdc3c7; text-decoration: none; }

main {
    display: flex;
    justify-content: center;
    padding: 3rem 1rem;
}

.card {
    background: white;
    border-radius: 12px;
    padding: 2rem;
    width: 100%;
    max-width: 480px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

.card h2 { margin-bottom: 1.5rem; color: #2c3e50; }

label { display: block; margin-bottom: 0.4rem; font-weight: bold; }

input[type="text"] {
    width: 100%;
    padding: 0.6rem 0.8rem;
    border: 1px solid #ccc;
```

```
    border-radius: 6px;
    font-size: 1rem;
    margin-bottom: 1rem;
}

button {
    width: 100%;
    padding: 0.7rem;
    background: #3498db;
    color: white;
    border: none;
    border-radius: 6px;
    font-size: 1rem;
    cursor: pointer;
}

button:hover { background: #2980b9; }

.message {
    background: #f0f8ff;
    border-left: 4px solid #3498db;
    padding: 1rem;
    border-radius: 4px;
    margin-bottom: 1.5rem;
    line-height: 1.7;
}
```

4단계: 실행 및 확인

```
# hello_ai 디렉토리에서 실행
python app.py
```

브라우저에서 `http://127.0.0.1:5000`으로 접속합니다. 이름 입력 후 "인사하기" 버튼을 클릭하면 AI 스타일의 인사 메시지가 나타납니다.

> ***실습 확장 아이디어***: *`greet()` 함수 내부에서 실제 AI 모델(`transformers`, `openai` 등)을 호출해 동적인 응답을 생성해 보세요. 이것이 바로 다음 챕터에서 다루는* ***AI 기반 웹앱****의 핵심입니다.*

정리

이번 챕터에서 학습한 내용을 정리합니다.

주제	핵심 내용
웹 기본 구조	클라이언트-서버 모델, HTTP 메서드, 상태 코드, 프론트엔드/백엔드
Flask 기초	`Flask(__name__)`, `app.run(debug=True)`, `@app.route()`
라우팅	URL 변수, HTTP 메서드 지정, `url_for()`
Jinja2 템플릿	`{{ }}`, `{% %}`, `extends`, `block`, `render_template()`
폼 처리	`request.form.get()`, GET/POST 구분
정적 파일	`static/` 폴더, `url_for('static', filename= ... )`

- 이전 챕터: [Chapter 20. RAG (검색 증강 생성)]
- 다음 챕터: [Chapter 22. AI 기반 웹앱 개발]

이번 챕터에서는 앞서 배운 Flask 웹 개발 기초(Ch.21)와 LLM API 활용 기술을 결합하여, 실제 현장에서 쓸 수 있는 **AI 기반 웹 애플리케이션(AI-powered Web Application)**을 처음부터 끝까지 완성합니다. 요구사항 분석부터 아키텍처 설계, 백엔드 API 연동, 엑셀 데이터 분석, 검색 통합, 챗봇 인터페이스, 보안 처리까지 실무 전 과정을 단계별로 다룹니다.

> *이 챕터를 마치면 비즈니스 로직, 엑셀 분석, 웹·DB 검색, 대화형 챗봇을 하나의 웹 앱에 통합할 수 있습니다.*

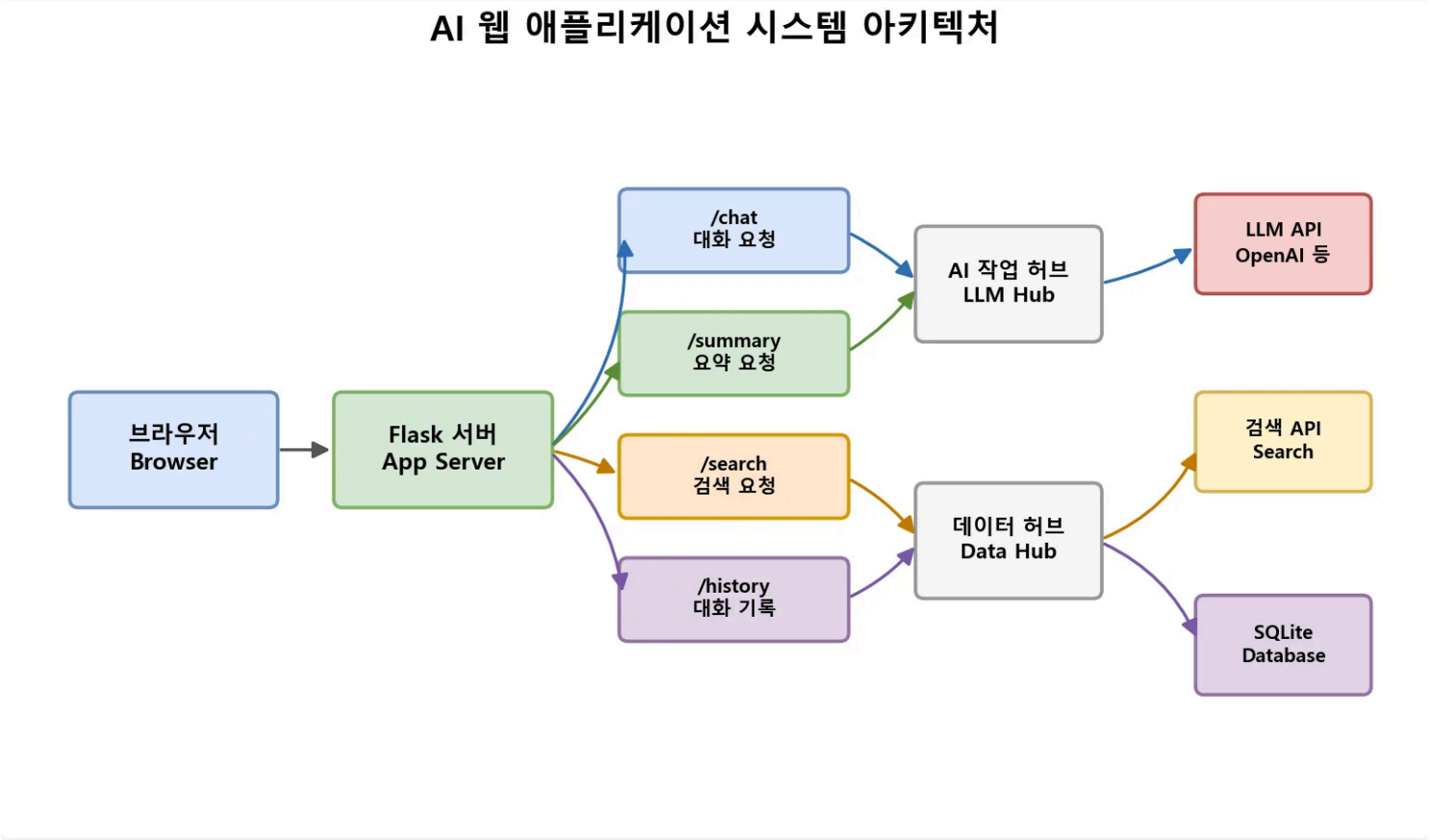
22.1 프로젝트 설계 — 요구사항 분석과 아키텍처

무엇을 만들 것인가

이번 챕터의 목표 애플리케이션은 **비즈니스 인텔리전스 웹앱(Business Intelligence Web App)**입니다. 주요 기능은 다음과 같습니다.

기능	설명
비즈니스 로직 API	사용자 질문 → LLM 답변 생성
엑셀 업로드 분석	.xlsx 파일 업로드 → AI 데이터 분석 요약
검색 통합	웹 검색 + 내부 DB 검색 결합 결과 반환
챗봇 인터페이스	다중 턴 대화(Multi-turn Conversation) 지원
보안 처리	API 키 보호, 입력 검증, XSS 방지

시스템 아키텍처



브라우저 클라이언트가 HTTP/WebSocket으로 Flask 웹 서버를 호출하면, Flask는 4개의 엔드포인트(`/api/chat` , `/api/analyze` , `/api/search` , `/result`)로 라우팅합니다. 각 엔드포인트는 서비스 레이어를 거쳐 **LLM API**, **SQLite DB**, **외부 검색 API**라는 3가지 외부 자원과 통신합니다.

디렉터리 구조

```
bi_webapp/
├── app.py                  # Flask 진입점
├── config.py               # 환경 변수 및 설정
├── .env                    # API 키 (Git 제외)
├── requirements.txt
├── services/
│   ├── llm_service.py      # LLM 호출 로직
│   ├── search_service.py   # 검색 통합 로직
│   └── excel_service.py    # 엑셀 파싱·분석 로직
├── models/
│   └── database.py         # SQLite ORM
├── static/
│   ├── css/style.css
│   └── js/chat.js
└── templates/
    ├── base.html
    ├── index.html
    └── result.html
```

> ***설계 원칙**: 비즈니스 로직은 `services/` 레이어에 격리하고, Flask 라우트는 HTTP 요청/응답 처리만 담당합니다. 이를 **관심사 분리(Separation of Concerns)**라고 합니다.*

22.2 비즈니스 로직 API 연동 백엔드 구현

환경 설정

```
# config.py
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    SECRET_KEY = os.environ.get("SECRET_KEY", "dev-secret-key")
    OPENAI_API_KEY = os.environ.get("OPENAI_API_KEY")
    ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY")
    SERPAPI_KEY = os.environ.get("SERPAPI_KEY")
    DATABASE_URI = os.environ.get("DATABASE_URI", "sqlite:///app.db")
    MAX_UPLOAD_SIZE_MB = 10
    ALLOWED_EXTENSIONS = {"xlsx", "xls", "csv"}
```

LLM 서비스 레이어

LLM 서비스(LLM Service)는 외부 AI API 호출을 추상화하여 라우트 코드가 특정 제공자에 종속되지 않도록 합니다.

```python
# services/llm_service.py
from openai import OpenAI
from config import Config

client = OpenAI(api_key=Config.OPENAI_API_KEY)

def ask_llm(system_prompt: str, user_message: str, history: list = None) → str:
    """LLM에 질문하고 응답 텍스트를 반환합니다."""
    messages = [{"role": "system", "content": system_prompt}]

    if history:
        # 최근 10턴만 컨텍스트로 전달 (토큰 절약)
        for turn in history[-10:]:
            messages.append({"role": turn["role"], "content": turn["content"]})

    messages.append({"role": "user", "content": user_message})

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=messages,
        max_tokens=1024,
        temperature=0.7,
    )
    return response.choices[0].message.content
```

Flask 라우트 구현

```
# app.py
from flask import Flask, request, jsonify, render_template, session
from services.llm_service import ask_llm
from services.search_service import hybrid_search
from services.excel_service import analyze_excel
from config import Config
import uuid

app = Flask(__name__)
app.config.from_object(Config)

BUSINESS_SYSTEM_PROMPT = """당신은 비즈니스 분석 전문가입니다.
데이터 기반의 명확하고 실행 가능한 인사이트를 제공합니다.
답변은 한국어로 작성하며, 핵심 내용을 먼저 제시합니다."""

@app.route("/")
def index():
    if "session_id" not in session:
        session["session_id"] = str(uuid.uuid4())
        session["history"] = []
    return render_template("index.html")

@app.route("/api/chat", methods=["POST"])
def chat():
    data = request.get_json()
    user_message = data.get("message", "").strip()

    if not user_message:
        return jsonify({"error": "메시지를 입력해 주세요."}), 400

    history = session.get("history", [])
    answer = ask_llm(BUSINESS_SYSTEM_PROMPT, user_message, history)

    # 대화 이력 업데이트
    history.append({"role": "user", "content": user_message})
    history.append({"role": "assistant", "content": answer})
    session["history"] = history

    return jsonify({"answer": answer})
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

> ***포인트**: `session` 에 대화 이력을 저장하면 서버 재시작 시 이력이 사라집니다. 영구 보존이 필요하면 DB에 저장하는 방식으로 확장합니다.*

22.3 결과 화면(Result Page) 구성

result.html 템플릿

결과 페이지(Result Page)는 AI 분석 결과를 시각적으로 보여주는 화면입니다. 마크다운 렌더링과 코드 하이라이팅을 지원합니다.

```
<!-- templates/result.html -->
{% extends "base.html" %}
{% block content %}
<div class="result-container">
  <div class="result-header">
    <h2>분석 결과</h2>
    <span class="badge">{{ analysis_type }}</span>
  </div>

  <div class="result-meta">
    <span>처리 시간: {{ elapsed_time }}초</span>
    <span>모델: {{ model_name }}</span>
  </div>

  <div class="result-body" id="result-content">
    {{ result_html | safe }}
  </div>

  <div class="result-actions">
    <button onclick="copyResult()">결과 복사</button>
    <a href="{{ url_for('index') }}">새 분석 시작</a>
  </div>
</div>

<script>
function copyResult() {
  const text = document.getElementById("result-content").innerText;
  navigator.clipboard.writeText(text).then(() => {
    alert("클립보드에 복사되었습니다.");
  });
}
</script>
{% endblock %}
```

마크다운 → HTML 변환

LLM 응답은 주로 마크다운 형식이므로, 서버 측에서 **marked.js** 또는 Python의 `markdown` 라이브러리를 사용해 HTML로 변환합니다.

```
# app.py 추가
import markdown

@app.route("/result")
def result_page():
    answer = request.args.get("answer", "")
    result_html = markdown.markdown(
        answer,
        extensions=["fenced_code", "tables", "nl2br"]
    )
    return render_template(
        "result.html",
        result_html=result_html,
        analysis_type=request.args.get("type", "일반 분석"),
        elapsed_time=request.args.get("elapsed", "-"),
        model_name="GPT-4o mini",
    )
```

22.4 엑셀 데이터 업로드 및 AI 분석 기능

엑셀 서비스 구현

pandas와 **openpyxl**을 사용하여 업로드된 파일을 파싱하고, 데이터 요약을 LLM에게 전달합니다.

```python
# services/excel_service.py
import pandas as pd
import io
from services.llm_service import ask_llm

EXCEL_SYSTEM_PROMPT = """당신은 데이터 분석 전문가입니다.
주어진 데이터 요약을 바탕으로 핵심 트렌드, 이상값, 비즈니스 인사이트를 제공합니다."""

def analyze_excel(file_bytes: bytes, filename: str, user_question: str) → dict:
    """엑셀 파일을 분석하고 AI 인사이트를 반환합니다."""
    try:
        df = pd.read_excel(io.BytesIO(file_bytes))
    except Exception:
        df = pd.read_csv(io.BytesIO(file_bytes))

    # 데이터 요약 생성
    summary = _build_summary(df)

    prompt = f"""다음 데이터를 분석해 주세요.

[데이터 개요]
{summary}

[사용자 질문]
{user_question}
"""
    answer = ask_llm(EXCEL_SYSTEM_PROMPT, prompt)
    return {
        "answer": answer,
        "rows": len(df),
        "columns": list(df.columns),
        "summary": summary,
    }

def _build_summary(df: pd.DataFrame) → str:
    """데이터프레임 요약 문자열을 생성합니다."""
    lines = [
        f"행 수: {len(df)}, 열 수: {len(df.columns)}",
        f"컬럼: {', '.join(df.columns.astype(str))}",
        "",
        "수치형 통계:",
        df.describe().to_string(),
```

```
        "",
        "결측값:",
        df.isnull().sum().to_string(),
        "",
        "처음 3행 미리보기:",
        df.head(3).to_string(),
    ]
    return "\n".join(lines)
```

업로드 라우트

```
# app.py 추가
from werkzeug.utils import secure_filename
import time

def allowed_file(filename: str) → bool:
    ext = filename.rsplit(".", 1)[-1].lower() if "." in filename else ""
    return ext in Config.ALLOWED_EXTENSIONS

@app.route("/api/analyze", methods=["POST"])
def analyze():
    if "file" not in request.files:
        return jsonify({"error": "파일이 없습니다."}), 400

    file = request.files["file"]
    question = request.form.get("question", "이 데이터의 핵심 인사이트를 알려주세요.")

    if not allowed_file(file.filename):
        return jsonify({"error": "xlsx, xls, csv 파일만 허용됩니다."}), 400

    # 파일 크기 검증
    file_bytes = file.read()
    if len(file_bytes) > Config.MAX_UPLOAD_SIZE_MB * 1024 * 1024:
        return jsonify({"error": f"파일 크기는 {Config.MAX_UPLOAD_SIZE_MB}MB 이하여야 합니
다."}), 413

    start = time.time()
    result = analyze_excel(file_bytes, secure_filename(file.filename), question)
    result["elapsed"] = round(time.time() - start, 2)

    return jsonify(result)
```

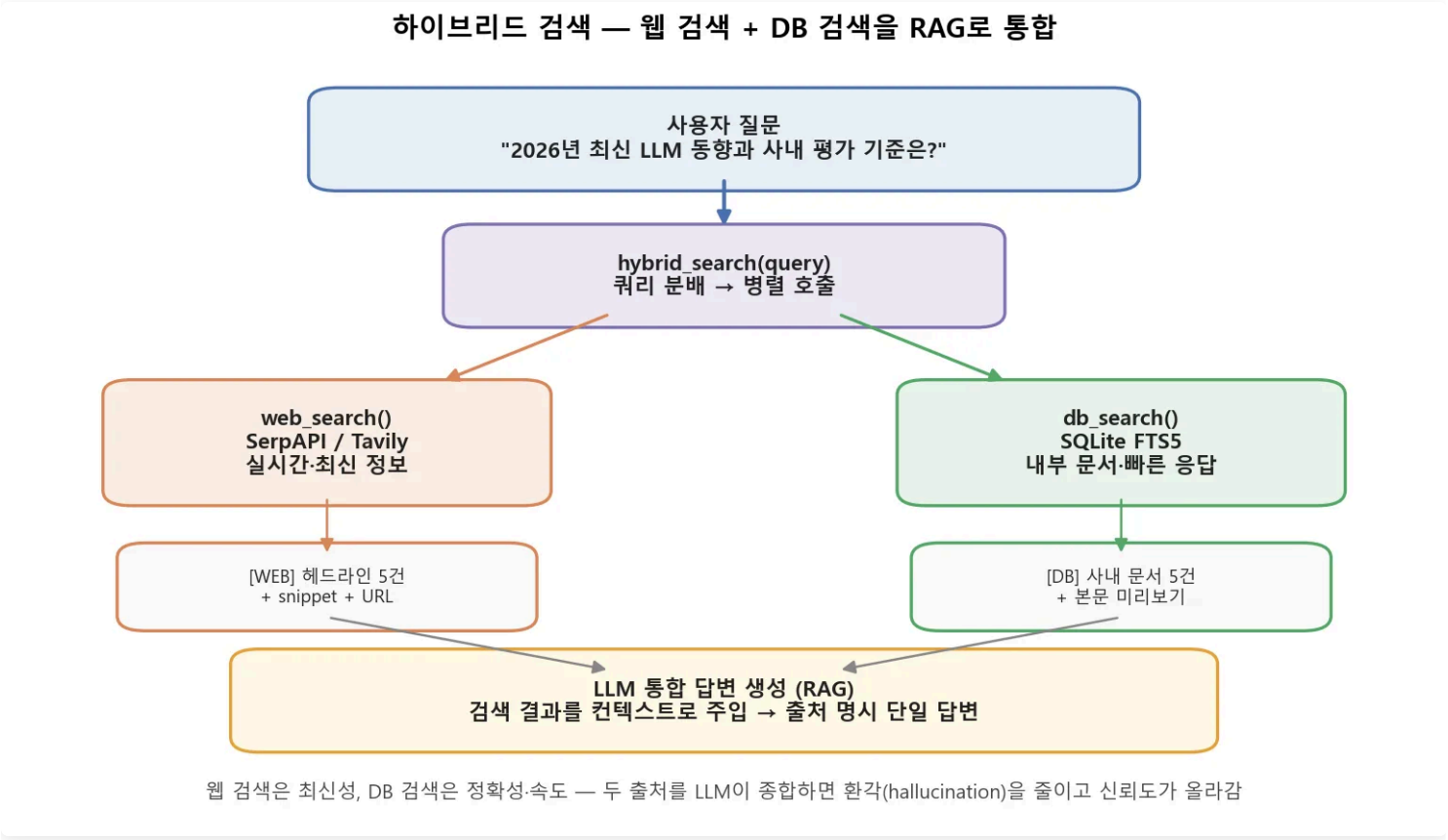
> ***주의***: *대용량 엑셀 파일(수십만 행)은 LLM 컨텍스트 창을 초과할 수 있습니다.* `_build_summary` *함수에서 통계 요약만 전달하는 방식이 안전합니다.*

22.5 검색 기능 통합 (웹검색 + DB검색)

하이브리드 검색 전략

하이브리드 검색(Hybrid Search)은 실시간 웹 검색 결과와 내부 DB 검색 결과를 합쳐 LLM에게 컨텍스트로 제공하는 **RAG(Retrieval-Augmented Generation)** 패턴입니다.

검색 유형	도구	특징
웹 검색	SerpAPI / Tavily	최신 정보, 실시간
DB 검색	SQLite FTS5	내부 문서, 빠른 속도
결합	LLM 요약	두 결과를 통합 답변



웹 검색은 최신성, DB 검색은 정확성·속도 — 두 출처를 LLM이 종합하면 환각(hallucination)을 줄이고 신뢰도가 올라감

`hybrid_search()`는 사용자 쿼리를 받아 `web_search()`(SerpAPI/Tavily)와 `db_search()`(SQLite FTS5)를 병렬로 호출하고, 두 결과를 묶어 LLM에게 컨텍스트로 전달합니다. 최신성(웹)과 정확성·속도(DB)를 동시에 확보해 환각을 줄이는 것이 핵심입니다.

검색 서비스 구현

```
# services/search_service.py
import requests
import sqlite3
from services.llm_service import ask_llm
from config import Config

SEARCH_SYSTEM_PROMPT = """당신은 검색 결과를 분석하는 전문가입니다.
웹 검색 결과와 내부 문서를 종합하여 정확하고 신뢰할 수 있는 답변을 제공합니다.
출처를 명시하고, 최신 정보를 우선합니다."""

def web_search(query: str, num_results: int = 5) → list[dict]:
    """SerpAPI를 사용한 웹 검색입니다."""
    params = {
        "q": query,
        "api_key": Config.SERPAPI_KEY,
        "num": num_results,
        "hl": "ko",
    }
    resp = requests.get("https://serpapi.com/search", params=params, timeout=10)
    resp.raise_for_status()
    data = resp.json()

    results = []
    for item in data.get("organic_results", []):
        results.append({
            "title": item.get("title", ""),
            "snippet": item.get("snippet", ""),
            "link": item.get("link", ""),
            "source": "web",
        })
    return results

def db_search(query: str, db_path: str = "app.db") → list[dict]:
    """SQLite FTS5 전문 검색입니다."""
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row
    try:
        rows = conn.execute(
            "SELECT title, content, created_at FROM documents_fts WHERE documents_fts MATCH ?
LIMIT 5",
```

```
            (query,),
        ).fetchall()
        return [
            {"title": r["title"], "snippet": r["content"][:300], "source": "db"}
            for r in rows
        ]
    except sqlite3.OperationalError:
        # FTS 테이블이 없는 경우 빈 결과 반환
        return []
    finally:
        conn.close()

def hybrid_search(query: str) → dict:
    """웹 검색 + DB 검색 결과를 LLM으로 통합합니다."""
    web_results = web_search(query)
    db_results = db_search(query)
    all_results = web_results + db_results

    if not all_results:
        return {"answer": "검색 결과를 찾을 수 없습니다.", "sources": []}

    context = "\n\n".join(
        f"[{r['source'].upper()}] {r['title']}\n{r['snippet']}"
        for r in all_results
    )
    prompt = f"""다음 검색 결과를 바탕으로 질문에 답변해 주세요.

[검색 결과]
{context}

[질문]
{query}
"""
    answer = ask_llm(SEARCH_SYSTEM_PROMPT, prompt)
    return {
        "answer": answer,
        "sources": [{"title": r["title"], "link": r.get("link", ""), "source": r["source"]} fo
r r in all_results],
    }
```

검색 라우트

```
# app.py 추가
@app.route("/api/search", methods=["POST"])
def search():
    data = request.get_json()
    query = data.get("query", "").strip()

    if not query:
        return jsonify({"error": "검색어를 입력해 주세요."}), 400
    if len(query) > 500:
        return jsonify({"error": "검색어는 500자 이하여야 합니다."}), 400

    result = hybrid_search(query)
    return jsonify(result)
```

22.6 대화형 챗봇 인터페이스 구현

프론트엔드 챗 UI

다중 턴 대화(Multi-turn Conversation) 인터페이스를 순수 HTML/CSS/JavaScript로 구현합니다.

```
<!-- templates/index.html -->
{% extends "base.html" %}
{% block content %}
<div class="chat-container">
  <div class="chat-header">
    <h1>AI 비즈니스 어시스턴트</h1>
    <button id="clear-btn" onclick="clearHistory()">대화 초기화</button>
  </div>

  <div class="chat-messages" id="chat-messages">
    <div class="message assistant">
      안녕하세요! 비즈니스 분석, 엑셀 데이터 분석, 검색 기능을 제공합니다. 무엇이든 물어보세요.
    </div>
  </div>

  <div class="chat-input-area">
    <!-- 파일 업로드 영역 -->
    <div class="upload-zone" id="upload-zone">
      <input type="file" id="file-input" accept=".xlsx,.xls,.csv" hidden>
      <span id="file-label" onclick="document.getElementById('file-input').click()">
        📎 엑셀 파일 첨부
      </span>
    </div>

    <div class="input-row">
      <textarea id="user-input" placeholder="메시지를 입력하세요..." rows="2"></textarea>
      <button id="send-btn" onclick="sendMessage()">전송</button>
    </div>
  </div>
</div>

<script src="{{ url_for('static', filename='js/chat.js') }}"></script>
{% endblock %}
```

챗 JavaScript (chat.js)

```
// static/js/chat.js

let attachedFile = null;

document.getElementById("file-input").addEventListener("change", function () {
  attachedFile = this.files[0];
  document.getElementById("file-label").textContent = `📎 ${attachedFile.name}`;
});

document.getElementById("user-input").addEventListener("keydown", function (e) {
  if (e.key === "Enter" && !e.shiftKey) {
    e.preventDefault();
    sendMessage();
  }
});

async function sendMessage() {
  const input = document.getElementById("user-input");
  const message = input.value.trim();
  if (!message && !attachedFile) return;

  appendMessage("user", message || `[파일 분석] ${attachedFile.name}`);
  input.value = "";
  setLoading(true);

  try {
    let data;
    if (attachedFile) {
      data = await analyzeFile(attachedFile, message || "이 데이터를 분석해 주세요.");
      attachedFile = null;
      document.getElementById("file-label").textContent = "📎 엑셀 파일 첨부";
    } else {
      data = await chatWithAI(message);
    }
    appendMessage("assistant", data.answer || data.error);
  } catch (err) {
    appendMessage("assistant", "오류가 발생했습니다. 다시 시도해 주세요.");
  } finally {
    setLoading(false);
  }
```

```
}

async function chatWithAI(message) {
  const resp = await fetch("/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message }),
  });
  return resp.json();
}

async function analyzeFile(file, question) {
  const formData = new FormData();
  formData.append("file", file);
  formData.append("question", question);
  const resp = await fetch("/api/analyze", { method: "POST", body: formData });
  return resp.json();
}

function appendMessage(role, text) {
  const container = document.getElementById("chat-messages");
  const div = document.createElement("div");
  div.className = `message ${role}`;
  div.innerHTML = marked.parse(text); // marked.js로 마크다운 렌더링
  container.appendChild(div);
  container.scrollTop = container.scrollHeight;
}

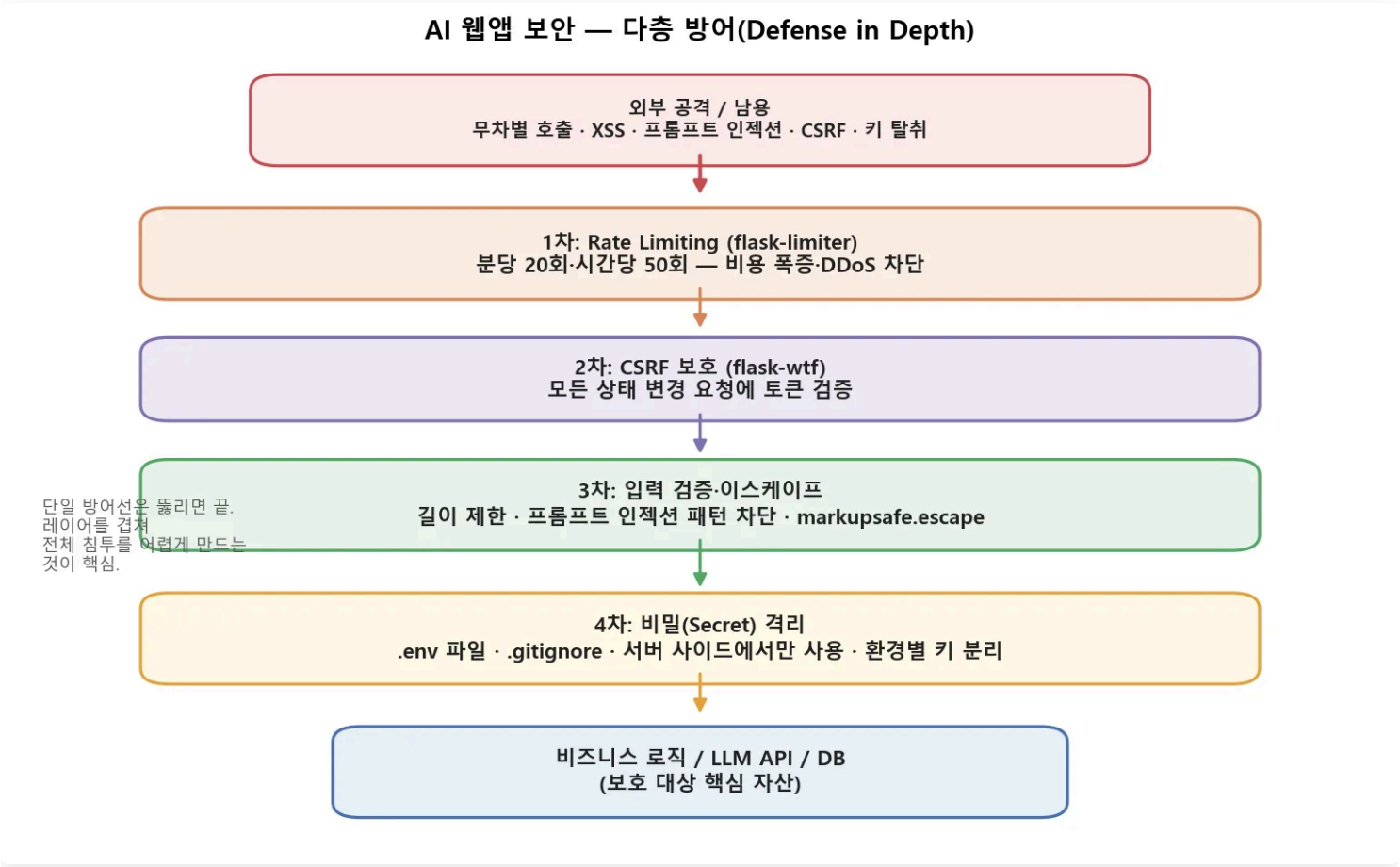
function setLoading(state) {
  document.getElementById("send-btn").disabled = state;
  document.getElementById("send-btn").textContent = state ? "처리 중..." : "전송";
}

async function clearHistory() {
  await fetch("/api/clear", { method: "POST" });
  document.getElementById("chat-messages").innerHTML =
    '<div class="message assistant">대화가 초기화되었습니다.</div>';
}
```

대화 초기화 라우트

```
# app.py 추가
@app.route("/api/clear", methods=["POST"])
def clear_history():
    session["history"] = []
    return jsonify({"status": "ok"})
```

22.7 보안 고려사항 (API Key 관리, 입력 검증)



보안은 단일 방어선이 아니라 **다층 방어(Defense in Depth)** 로 설계합니다. 외부 공격이 Rate Limiting → CSRF → 입력 검증 → 비밀(Secret) 격리 4개 레이어를 순차적으로 통과해야 핵심 비즈니스 로직과 LLM API에 닿을 수 있도록 구성하면, 한 레이어가 뚫려도 전체 시스템이 즉시 무너지지 않습니다.

API 키 관리 원칙

API 키(API Key)는 애플리케이션에서 가장 민감한 자산입니다. 다음 원칙을 반드시 지킵니다.

나쁜 예	좋은 예
코드에 직접 하드코딩	`.env` 파일로 분리
Git에 커밋	`.gitignore` 에 `.env` 추가
프론트엔드 JavaScript에 노출	서버 사이드에서만 사용
단일 키 공유	환경별(개발/운영) 키 분리

```
# .gitignore 필수 항목
.env
*.env
secrets/
```

```
# .env 예시 (절대 커밋 금지)
OPENAI_API_KEY=sk-...
SERPAPI_KEY=...
SECRET_KEY=random-32-char-string
```

입력 검증 미들웨어

입력 검증(Input Validation)은 **XSS(Cross-Site Scripting)**, **프롬프트 인젝션(Prompt Injection)**, 과도한 API 사용을 막는 핵심 방어선입니다.

```
# app.py 추가 — 요청 전 공통 검증
import re
from functools import wraps
from markupsafe import escape

# 허용되지 않는 패턴 (프롬프트 인젝션 시도)
INJECTION_PATTERNS = [
    r"ignore previous instructions",
    r"system prompt",
    r"act as",
    r"jailbreak",
]

def validate_input(text: str, max_length: int = 2000) → tuple[bool, str]:
    """입력 텍스트를 검증하고 (유효여부, 오류메시지)를 반환합니다."""
    if not text or not text.strip():
        return False, "입력값이 비어 있습니다."
    if len(text) > max_length:
        return False, f"입력값은 {max_length}자 이하여야 합니다."

    lower = text.lower()
    for pattern in INJECTION_PATTERNS:
        if re.search(pattern, lower):
            return False, "허용되지 않는 표현이 포함되어 있습니다."

    return True, ""

def require_json(f):
    """JSON Content-Type을 강제하는 데코레이터입니다."""
    @wraps(f)
    def decorated(*args, **kwargs):
        if not request.is_json:
            return jsonify({"error": "Content-Type은 application/json이어야 합니다."}), 415
        return f(*args, **kwargs)
    return decorated
```

레이트 리미팅(Rate Limiting)

레이트 리미팅(Rate Limiting)으로 API 남용과 비용 폭증을 방지합니다.

```
# flask-limiter 패키지 사용
# pip install flask-limiter

from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    app=app,
    key_func=get_remote_address,
    default_limits=["200 per day", "50 per hour"],
)

@app.route("/api/chat", methods=["POST"])
@limiter.limit("20 per minute")  # 분당 20회 제한
def chat():
    ...
```

CSRF 보호

CSRF(Cross-Site Request Forgery) 공격을 방지하려면 `flask-wtf`의 CSRF 보호를 활성화합니다.

```
# pip install flask-wtf
from flask_wtf.csrf import CSRFProtect

csrf = CSRFProtect(app)
```

```
<!-- 모든 AJAX 요청에 CSRF 토큰 포함 -->
<meta name="csrf-token" content="{{ csrf_token() }}">

<script>
const csrfToken = document.querySelector('meta[name="csrf-token"]').content;

fetch("/api/chat", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "X-CSRFToken": csrfToken,
  },
  body: JSON.stringify({ message }),
});
</script>
```

[실습] AI 비즈니스 웹앱 완성 (비즈니스 로직 + 엑셀 + 검색 통합)

실습 목표

지금까지 구현한 모든 기능을 하나의 Flask 앱으로 통합하여 실제 구동되는 AI 비즈니스 웹앱을 완성합니다.

환경 준비

```
# 가상환경 생성 및 패키지 설치
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

pip install flask flask-wtf flask-limiter \
            openai pandas openpyxl markdown \
            python-dotenv requests markupsafe
```

최종 app.py 통합 코드

```
# app.py (최종 통합본)
from flask import Flask, request, jsonify, render_template, session
from flask_wtf.csrf import CSRFProtect
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address
from werkzeug.utils import secure_filename
from markupsafe import escape
import markdown
import uuid
import time

from config import Config
from services.llm_service import ask_llm
from services.search_service import hybrid_search
from services.excel_service import analyze_excel

app = Flask(__name__)
app.config.from_object(Config)
csrf = CSRFProtect(app)
limiter = Limiter(app=app, key_func=get_remote_address,
                  default_limits=["200 per day", "50 per hour"])

BUSINESS_SYSTEM_PROMPT = """당신은 비즈니스 분석 전문가입니다.
데이터 기반의 명확하고 실행 가능한 인사이트를 제공합니다."""

def allowed_file(filename):
    return "." in filename and \
           filename.rsplit(".", 1)[1].lower() in Config.ALLOWED_EXTENSIONS

@app.route("/")
def index():
    if "session_id" not in session:
        session["session_id"] = str(uuid.uuid4())
        session["history"] = []
    return render_template("index.html")

@app.route("/api/chat", methods=["POST"])
@limiter.limit("20 per minute")
def chat():
    data = request.get_json()
```

```
    message = data.get("message", "").strip()
    if not message:
        return jsonify({"error": "메시지를 입력해 주세요."}), 400
    if len(message) > 2000:
        return jsonify({"error": "메시지는 2000자 이하여야 합니다."}), 400

    history = session.get("history", [])
    answer = ask_llm(BUSINESS_SYSTEM_PROMPT, message, history)
    history.append({"role": "user", "content": message})
    history.append({"role": "assistant", "content": answer})
    session["history"] = history
    return jsonify({"answer": answer})

@app.route("/api/analyze", methods=["POST"])
@limiter.limit("10 per minute")
def analyze():
    if "file" not in request.files:
        return jsonify({"error": "파일이 없습니다."}), 400
    file = request.files["file"]
    question = request.form.get("question", "이 데이터의 핵심 인사이트를 알려주세요.")

    if not allowed_file(file.filename):
        return jsonify({"error": "xlsx, xls, csv 파일만 허용됩니다."}), 400

    file_bytes = file.read()
    if len(file_bytes) > Config.MAX_UPLOAD_SIZE_MB * 1024 * 1024:
        return jsonify({"error": f"{Config.MAX_UPLOAD_SIZE_MB}MB 이하 파일만 허용됩니다."}), 41
3

    start = time.time()
    result = analyze_excel(file_bytes, secure_filename(file.filename), question)
    result["elapsed"] = round(time.time() - start, 2)
    return jsonify(result)

@app.route("/api/search", methods=["POST"])
@limiter.limit("15 per minute")
def search():
    data = request.get_json()
    query = data.get("query", "").strip()
    if not query:
        return jsonify({"error": "검색어를 입력해 주세요."}), 400
    if len(query) > 500:
        return jsonify({"error": "검색어는 500자 이하여야 합니다."}), 400
```

```
    result = hybrid_search(query)
    return jsonify(result)

@app.route("/api/clear", methods=["POST"])
def clear_history():
    session["history"] = []
    return jsonify({"status": "ok"})

@app.route("/result")
def result_page():
    answer = request.args.get("answer", "")
    result_html = markdown.markdown(
        answer, extensions=["fenced_code", "tables", "nl2br"]
    )
    return render_template("result.html", result_html=result_html,
                           analysis_type=request.args.get("type", "일반 분석"),
                           elapsed_time=request.args.get("elapsed", "-"),
                           model_name="GPT-4o mini")

if __name__ == "__main__":
    app.run(debug=False)  # 운영 환경에서는 debug=False 필수
```

실행 및 테스트

```
# 1. 환경 변수 설정
cp .env.example .env
# .env 파일에 실제 API 키 입력

# 2. 서버 실행
python app.py

# 3. 브라우저에서 확인
# http://127.0.0.1:5000
```

기능별 동작 확인 체크리스트

- [ ] 텍스트 메시지 입력 → AI 답변 출력
- [ ] 엑셀 파일 첨부 후 질문 → 데이터 분석 결과 출력
- [ ] 검색 탭에서 쿼리 입력 → 웹+DB 통합 결과 출력
- [ ] 대화 초기화 버튼 → 이전 이력 삭제 확인

- [ ] 2000자 초과 입력 → 오류 메시지 출력 확인
- [ ] `.env` 파일 없이 실행 → 설정 오류 메시지 확인

핵심 정리

이번 챕터에서 학습한 내용을 정리합니다.

주제	핵심 내용
아키텍처	관심사 분리(서비스 레이어 분리), MVC 패턴
LLM 연동	시스템 프롬프트, 다중 턴 이력 관리
엑셀 분석	pandas 요약 → LLM 컨텍스트 전달
하이브리드 검색	웹 검색 + DB 검색 → RAG 패턴
챗봇 UI	marked.js 마크다운 렌더링, 비동기 fetch
보안	.env 분리, 입력 검증, Rate Limiting, CSRF 보호

> *다음 챕터(Ch.23)에서는 완성된 이 웹앱을 클라우드에 배포하고, 도메인 연결과 HTTPS 설정, 컨테이너화(Docker) 방법을 학습합니다.*

이 챕터에서는 완성된 AI 웹 애플리케이션을 실제 사용자에게 공개하는 방법을 학습합니다. 로컬 개발 환경에서 벗어나 외부에서 접근 가능한 서비스로 만드는 전 과정, 즉 **배포(Deployment)**의 핵심 개념부터 실전 클라우드 배포까지 단계적으로 다룹니다.

> *이전 챕터(Ch.22)에서 완성한 AI 기반 웹앱을 실제로 세상에 공개해 봅니다. 이 챕터를 마치면 누구나 URL 하나로 여러분의 앱에 접근할 수 있게 됩니다.*

23.1 배포 개요 — 로컬 vs 클라우드

로컬 환경의 한계

지금까지 `python app.py`를 실행하면 `http://127.0.0.1:5000` 처럼 **로컬호스트(localhost)**에서만 앱이 동작했습니다. 이 주소는 내 컴퓨터 안에서만 유효하며, 다른 사람은 접근할 수 없습니다.

배포(Deployment)란 애플리케이션을 외부 서버에 올려서 인터넷을 통해 누구나 접근할 수 있도록 하는 작업입니다.

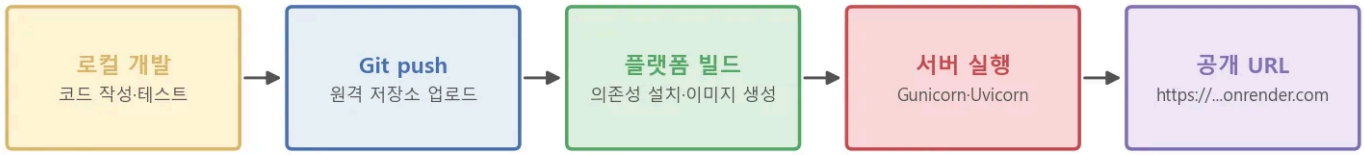
배포 방식 비교

구분	로컬 실행	ngrok 터널링	클라우드 배포
접근 범위	내 컴퓨터만	임시 공개 URL	전 세계
지속성	프로세스 종료 시 중단	터널 세션 동안만	24시간 상시 운영
비용	무료	무료(기본)	무료 플랜 있음
난이도	매우 쉬움	쉬움	보통
사용 목적	개발·테스트	데모·테스트 공유	실제 서비스 운영

배포 흐름 개요

일반적인 웹 애플리케이션 배포 흐름은 다음과 같습니다.

클라우드 배포 흐름 — 로컬 코드에서 공개 URL까지



WSGI 서버(Gunicorn) + .env 환경변수 + 플랫폼 자동 빌드의 조합으로 누구나 접속 가능한 서비스가 됩니다.

WSGI(Web Server Gateway Interface)는 Python 웹 프레임워크(Flask, Django 등)와 웹 서버 사이를 연결하는 표준 인터페이스입니다. 클라우드 배포 시 Flask 내장 서버 대신 **Gunicorn** 같은 WSGI 서버를 사용하는 이유가 여기 있습니다.

23.2 ngrok을 이용한 터널링과 임시 공개

ngrok이란?

ngrok(엔그록)은 로컬에서 실행 중인 서버를 외부 인터넷에 임시로 공개할 수 있게 해주는 **터널링(Tunneling)** 도구입니다. 별도 서버 없이도 `https://xxxx.ngrok-free.app` 같은 URL을 즉시 받을 수 있습니다.

> *데모 발표, 팀원 테스트, 웹훅(Webhook) 개발 시 매우 유용합니다.*

ngrok 설치 및 설정

```
# pip로 파이썬 래퍼 설치
pip install pyngrok
```

공식 사이트(https://ngrok.com)에서 무료 계정을 만들고 **인증 토큰(Authtoken)**을 발급받습니다.

```
# 인증 토큰 등록 (한 번만 실행)
ngrok config add-authtoken YOUR_AUTHTOKEN
```

ngrok으로 로컬 Flask 앱 공개하기

방법 1: 커맨드라인에서 직접 실행

Flask 앱을 먼저 실행한 뒤, 별도 터미널에서 ngrok을 실행합니다.

```
# 터미널 1: Flask 앱 실행
python app.py

# 터미널 2: ngrok 터널 생성
ngrok http 5000
```

실행하면 다음과 같은 화면이 나타납니다.

```
Session Status     online
Account            your@email.com
Forwarding         https://abcd-1234.ngrok-free.app → http://localhost:5000
```

`Forwarding` 줄의 `https://` 주소를 공유하면 누구나 접속할 수 있습니다.

방법 2: 파이썬 코드에서 pyngrok 사용

```
from flask import Flask
from pyngrok import ngrok

app = Flask(__name__)

@app.route("/")
def index():
    return "안녕하세요! ngrok으로 공개된 앱입니다."

if __name__ == "__main__":
    # ngrok 터널 생성
    public_url = ngrok.connect(5000)
    print(f"공개 URL: {public_url}")
    app.run(port=5000)
```

ngrok 사용 시 주의사항

- 무료 플랜은 **세션당 임시 URL**이 발급되며, 재시작하면 URL이 바뀝니다.
- 앱을 종료하면 터널도 함께 닫힙니다.
- 민감한 데이터가 포함된 앱은 임시 공개 후 반드시 터널을 종료하세요.
- **고정 도메인(Static Domain)**은 유료 플랜 또는 무료 계정 1개 한도로 제공됩니다.

23.3 클라우드 배포 기초 (Heroku / Railway / Render)

클라우드 플랫폼 비교

PaaS(Platform as a Service)는 서버 인프라 관리 없이 코드만 올리면 알아서 실행해 주는 클라우드 서비스 방식입니다. Flask 앱 배포에 많이 사용되는 대표 플랫폼을 비교합니다.

플랫폼	무료 플랜	특징	추천 대상
Render	무료 웹 서비스 제공(슬립 있음), 직관적 UI	GitHub 연동 간편, HTTPS 자동	포트폴리오·소규모 서비스
Railway	신규 가입 시 일정 크레딧 제공(이후 유료)	빠른 배포, GitHub 연동 간편	개인 프로젝트
Fly.io	일정 무료 한도(소형 VM·트래픽)	글로벌 엣지 배포, Docker 친화	글로벌 분산 데모
Hugging Face Spaces	무료 CPU 인스턴스 제공	Gradio/Streamlit/Docker, AI 데모 사실상 표준	AI 모델 데모
Vercel	서버리스 무료 한도	프런트엔드·서버리스 함수 강점	정적 + 경량 API
Heroku	**무료 플랜 없음(2022.11 종료, 전면 유료)**	가장 오랜 역사, 생태계 풍부	유료 운영이 가능한 팀 프로젝트

> ⚠️ ***Heroku 무료 dyno는 2022년 11월부로 종료****되었습니다. 현재는 모든 플랜이 유료이므로, 학습·포트폴리오 목적이라면 Render, Fly.io, Hugging Face Spaces 등의 무료 옵션부터 검토하는 것을 권장합니다.*
>
> *이 책에서는 **Render**를 기준으로 설명합니다. 무료 플랜이 있고 GitHub 연동이 간단하기 때문입니다. AI 모델 데모를 빠르게 공유하려는 경우에는 **Hugging Face Spaces**가 사실상 표준이므로 함께 익혀두면 유용합니다.*

배포 전 필수 파일 준비

클라우드 플랫폼은 저장소를 클론해 앱을 자동으로 빌드합니다. 다음 파일들이 필요합니다.

requirements.txt — 의존 패키지 목록

```
# 현재 환경의 패키지를 파일로 저장
pip freeze > requirements.txt
```

```
flask==3.0.3
gunicorn==22.0.0
openai==1.30.0
python-dotenv==1.0.1
```

Procfile — 앱 실행 명령 정의 (Heroku/Railway)

```
web: gunicorn app:app
```

> *`app:app` 은 `app.py` 파일의 `app` 객체를 의미합니다.*

runtime.txt — 파이썬 버전 명시 (선택)

```
python-3.11.9
```

Render 배포 단계별 가이드

1단계: GitHub에 코드 올리기

```
git init
git add .
git commit -m "initial commit"
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git push -u origin main
```

2단계: Render 서비스 생성

1. https://render.com 에서 회원가입 후 로그인합니다.
2. 대시보드에서 **New → Web Service**를 클릭합니다.
3. GitHub 저장소를 연결합니다.
4. 설정값을 입력합니다.

항목	값
Environment	Python 3
Build Command	`pip install -r requirements.txt`
Start Command	`gunicorn app:app`
Instance Type	Free

5. **Create Web Service**를 클릭하면 자동 빌드가 시작됩니다.

3단계: 배포 확인

빌드 로그에서 `Your service is live` 메시지가 나타나면 배포가 완료된 것입니다. 제공된 `https://your-app.onrender.com` URL로 접속합니다.

Gunicorn 설정

Flask 내장 서버는 개발용이므로 프로덕션에서는 **Gunicorn(그린 유니콘)**을 사용합니다.

```
# 로컬에서 Gunicorn으로 실행 테스트
gunicorn app:app --workers 2 --bind 0.0.0.0:5000
```

- `--workers 2` : 동시 요청을 처리하는 워커 프로세스 수
- `--bind 0.0.0.0:5000` : 모든 네트워크 인터페이스에서 5000번 포트 수신

23.4 환경변수와 시크릿 관리

왜 환경변수가 필요한가?

API 키, 데이터베이스 비밀번호 같은 민감한 정보를 코드에 직접 작성하면 GitHub에 공개될 위험이 있습니다.

```
# 절대 이렇게 하지 마세요!
openai.api_key = "sk-proj-abcdefg12345..."
```

환경변수(Environment Variable)는 운영체제 또는 배포 플랫폼에서 설정하는 키-값 쌍으로, 코드와 분리해 민감한 정보를 관리하는 표준 방식입니다.

로컬 환경: .env 파일 사용

```
# .env 파일 생성
OPENAI_API_KEY=sk-proj-your-key-here
SECRET_KEY=your-flask-secret-key
DATABASE_URL=sqlite:///app.db
```

`.gitignore`에 `.env`를 반드시 추가합니다.

```
# .gitignore
.env
__pycache__/
*.pyc
venv/
```

python-dotenv로 `.env` 파일을 로드합니다.

```
import os
from dotenv import load_dotenv

# .env 파일 로드 (로컬 개발용)
load_dotenv()

# 환경변수 읽기
openai_api_key = os.environ.get("OPENAI_API_KEY")
secret_key = os.environ.get("SECRET_KEY", "fallback-dev-key")

if not openai_api_key:
    raise ValueError("OPENAI_API_KEY 환경변수가 설정되지 않았습니다.")
```

클라우드 환경: 플랫폼 환경변수 설정

배포 플랫폼에서는 `.env` 파일 대신 대시보드에서 직접 환경변수를 설정합니다.

Render 환경변수 설정 방법:

1. Render 대시보드 → 해당 서비스 클릭
2. **Environment** 탭 선택
3. **Add Environment Variable** 클릭
4. Key: `OPENAI_API_KEY`, Value: 실제 API 키 입력
5. **Save Changes**

Railway 환경변수 설정 방법:

1. 프로젝트 → **Variables** 탭
2. `+ New Variable`로 추가

시크릿 관리 모범 사례

```
# config.py — 설정을 한 곳에서 관리
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    """기본 설정"""
    SECRET_KEY = os.environ.get("SECRET_KEY") or "dev-secret-key"
    OPENAI_API_KEY = os.environ.get("OPENAI_API_KEY")

class DevelopmentConfig(Config):
    """개발 환경 설정"""
    DEBUG = True

class ProductionConfig(Config):
    """프로덕션 환경 설정"""
    DEBUG = False

# 환경에 따라 설정 선택
config = {
    "development": DevelopmentConfig,
    "production": ProductionConfig,
    "default": DevelopmentConfig,
}
```

```
# app.py에서 사용
from config import config
import os

app = Flask(__name__)
env = os.environ.get("FLASK_ENV", "default")
app.config.from_object(config[env])
```

23.5 로깅과 모니터링 기초

로깅이 중요한 이유

로깅(Logging)은 애플리케이션의 동작 기록을 파일이나 콘솔에 남기는 것입니다. 배포 후 오류가 발생했을 때 원인을 추적하는 핵심 수단입니다.

> *`print()` 는 개발 중 디버깅에는 편리하지만, 프로덕션에서는 Python 표준 `logging` 모듈을 사용하는 것이 권장됩니다.*

Python logging 모듈 기본 사용법

로그 레벨(Log Level) 은 중요도에 따라 5단계로 구분됩니다.

레벨	숫자	용도
DEBUG	10	상세 디버깅 정보
INFO	20	정상 동작 확인
WARNING	30	주의가 필요한 상황
ERROR	40	기능 오류 발생
CRITICAL	50	심각한 시스템 오류

```
import logging
import os

# 로깅 설정
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    handlers=[
        logging.StreamHandler(),           # 콘솔 출력
        logging.FileHandler("app.log"),    # 파일 저장
    ]
)

logger = logging.getLogger(__name__)
```

Flask 앱에 로깅 통합

```
from flask import Flask, request, jsonify
import logging
import time

app = Flask(__name__)
logger = logging.getLogger(__name__)

@app.before_request
def log_request_info():
    """모든 요청을 로그에 기록"""
    logger.info(f"요청: {request.method} {request.path} | IP: {request.remote_addr}")

@app.after_request
def log_response_info(response):
    """모든 응답을 로그에 기록"""
    logger.info(f"응답: {response.status_code}")
    return response

@app.errorhandler(Exception)
def handle_exception(e):
    """처리되지 않은 예외를 로그에 기록"""
    logger.error(f"처리되지 않은 오류: {str(e)}", exc_info=True)
    return jsonify({"error": "서버 내부 오류가 발생했습니다."}), 500
```

AI API 호출 로깅

AI 기능이 포함된 앱에서는 API 호출 현황을 추적하는 것이 중요합니다.

```
import time
import logging
from openai import OpenAI

logger = logging.getLogger(__name__)
client = OpenAI()

def call_openai_with_logging(prompt: str, model: str = "gpt-4o-mini") → str:
    """로깅이 포함된 OpenAI API 호출 함수"""
    start_time = time.time()

    try:
        logger.info(f"OpenAI 호출 시작 | 모델: {model} | 프롬프트 길이: {len(prompt)}")

        response = client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}]
        )

        elapsed = time.time() - start_time
        usage = response.usage

        logger.info(
            f"OpenAI 호출 완료 | 소요시간: {elapsed:.2f}s | "
            f"입력토큰: {usage.prompt_tokens} | 출력토큰: {usage.completion_tokens}"
        )

        return response.choices[0].message.content

    except Exception as e:
        elapsed = time.time() - start_time
        logger.error(f"OpenAI 호출 실패 | 소요시간: {elapsed:.2f}s | 오류: {str(e)}")
        raise
```

클라우드 플랫폼 로그 확인

클라우드 배포 후에는 플랫폼 대시보드에서 실시간 로그를 확인할 수 있습니다.

Render 로그 확인:

- 서비스 대시보드 → **Logs** 탭

Railway 로그 확인:

- 프로젝트 → 서비스 클릭 → **Deployments** → 최신 배포 → **View Logs**

로컬에서 실시간 로그 확인:

```
# Render CLI 사용 (설치 필요)
render logs --tail

# 로컬 파일 실시간 확인
tail -f app.log
```

모니터링 기초

업타임(Uptime) 모니터링은 서비스가 정상적으로 응답하는지 주기적으로 확인하는 것입니다. 간단한 헬스체크 엔드포인트를 만들어 두면 편리합니다.

```
from datetime import datetime, timezone

@app.route("/health")
def health_check():
    """서비스 상태 확인용 엔드포인트"""
    return jsonify({
        "status": "ok",
        # Python 3.12+: datetime.utcnow()는 deprecated이므로 timezone-aware 객체 사용
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "version": "1.0.0"
    }), 200
```

무료 업타임 모니터링 서비스(예: **UptimeRobot**)를 연결하면 서비스 다운 시 이메일 알림을 받을 수 있습니다.

[실습] ngrok으로 로컬 앱 외부 공개 + 클라우드 배포 체험

이번 실습에서는 Ch.22에서 만든 AI 웹앱을 두 가지 방법으로 외부에 공개합니다.

실습 목표

- ngrok으로 로컬 Flask 앱을 임시 공개하여 팀원과 공유하기
- Render에 클라우드 배포하여 상시 접근 가능한 URL 만들기

실습용 Flask 앱 준비

아래 파일 구조를 기준으로 진행합니다.

```
ch23_deploy/
├── app.py
├── requirements.txt
├── Procfile
├── .env
├── .gitignore
└── templates/
    └── index.html
```

app.py

```python
import os
import logging
from flask import Flask, render_template, request, jsonify
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s"
)
logger = logging.getLogger(__name__)

app = Flask(__name__)
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json()
    user_message = data.get("message", "")

    if not user_message:
        return jsonify({"error": "메시지를 입력하세요."}), 400

    logger.info(f"사용자 메시지: {user_message[:50]}")

    try:
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": user_message}]
        )
        reply = response.choices[0].message.content
        logger.info("응답 생성 완료")
        return jsonify({"reply": reply})
    except Exception as e:
        logger.error(f"API 오류: {str(e)}")
        return jsonify({"error": "AI 응답 생성 중 오류가 발생했습니다."}), 500
```

```
@app.route("/health")
def health():
    return jsonify({"status": "ok"}), 200

if __name__ == "__main__":
    app.run(debug=True)
```

templates/index.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <title>AI 챗봇 - Ch.23 실습</title>
    <style>
        body { font-family: sans-serif; max-width: 600px; margin: 40px auto; padding: 0 20px;
}
        #chat-box { border: 1px solid #ddd; height: 300px; overflow-y: auto; padding: 10px; ma
rgin-bottom: 10px; }
        .user-msg { color: #0066cc; margin: 5px 0; }
        .ai-msg { color: #333; margin: 5px 0; }
        input { width: 80%; padding: 8px; }
        button { padding: 8px 16px; cursor: pointer; }
    </style>
</head>
<body>
    <h1>AI 챗봇</h1>
    <div id="chat-box"></div>
    <input type="text" id="user-input" placeholder="메시지를 입력하세요..." />
    <button onclick="sendMessage()">전송</button>

    <script>
        async function sendMessage() {
            const input = document.getElementById("user-input");
            const chatBox = document.getElementById("chat-box");
            const message = input.value.trim();
            if (!message) return;

            chatBox.innerHTML += `<p class="user-msg">나: ${message}</p>`;
            input.value = "";

            const res = await fetch("/chat", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ message })
            });
            const data = await res.json();
            chatBox.innerHTML += `<p class="ai-msg">AI: ${data.reply || data.error}</p>`;
            chatBox.scrollTop = chatBox.scrollHeight;
        }
```

```
        document.getElementById("user-input").addEventListener("keypress", (e) => {
            if (e.key === "Enter") sendMessage();
        });
    </script>
</body>
</html>
```

requirements.txt

```
flask==3.0.3
gunicorn==22.0.0
openai==1.30.0
python-dotenv==1.0.1
pyngrok==7.2.0
```

Procfile

```
web: gunicorn app:app
```

.gitignore

```
.env
__pycache__/
*.pyc
venv/
*.log
```

STEP 1: ngrok으로 임시 공개

```
# 1. 패키지 설치
pip install -r requirements.txt

# 2. .env 파일에 API 키 설정
echo "OPENAI_API_KEY=sk-proj-your-key" > .env

# 3. Flask 앱 실행 (터미널 1)
python app.py

# 4. ngrok 터널 실행 (터미널 2)
ngrok http 5000
```

터미널 2에 출력된 `https://xxxx.ngrok-free.app` URL을 스마트폰이나 다른 사람에게 공유해 접속을 확인합니다.

STEP 2: GitHub에 코드 올리기

```
git init
git add app.py requirements.txt Procfile templates/ .gitignore
git commit -m "ch23: deploy practice app"

# GitHub에서 새 저장소 생성 후
git remote add origin https://github.com/YOUR_USERNAME/ch23-deploy.git
git push -u origin main
```

> *`.env` 파일은 절대 커밋하지 않도록 주의합니다. `.gitignore`에 포함되어 있지만 `git status`로 반드시 확인하세요.*

STEP 3: Render에 배포

1. https://render.com 로그인 → **New Web Service**
2. GitHub 저장소 연결 후 설정 입력: - Build Command: `pip install -r requirements.txt` - Start Command: `gunicorn app:app`
3. **Environment** 탭에서 `OPENAI_API_KEY` 환경변수 추가
4. **Create Web Service** 클릭

빌드 완료 후 `https://ch23-deploy.onrender.com` 형태의 URL로 접속합니다.

STEP 4: 배포 후 로그 확인

Render 대시보드 → **Logs** 탭에서 실시간 로그를 확인합니다. 앱에 접속하거나 채팅을 시도하면 로그가 실시간으로 출력되는 것을 볼 수 있습니다.

```
2026-04-01 12:00:01 [INFO] 요청: POST /chat | IP: 203.xxx.xxx.xxx
2026-04-01 12:00:01 [INFO] 사용자 메시지: 안녕하세요
2026-04-01 12:00:02 [INFO] 응답 생성 완료
2026-04-01 12:00:02 [INFO] 응답: 200
```

실습 결과 정리

단계	완료 기준
ngrok 터널링	외부 URL로 챗봇 접속 성공
GitHub push	저장소에 코드 업로드 완료
Render 배포	`onrender.com` URL로 접속 성공
로그 확인	대시보드에서 요청 로그 확인

핵심 정리

- **배포**는 로컬 앱을 외부 서버에 올려 인터넷으로 접근 가능하게 만드는 작업입니다.
- **ngrok**은 별도 서버 없이 로컬 앱을 임시 공개할 수 있는 터널링 도구입니다.
- **PaaS 플랫폼**(Render, Railway, Heroku)은 코드만 올리면 자동으로 빌드·배포해 줍니다.
- **환경변수**로 API 키 등 민감한 정보를 코드와 분리해 관리합니다. `.env` 파일은 `.gitignore`에 추가하세요.
- **로깅**은 배포 후 오류 추적의 핵심 수단입니다. Python `logging` 모듈을 활용하세요.
- 프로덕션 환경에서는 Flask 내장 서버 대신 **Gunicorn**을 사용합니다.

*다음 챕터(Ch.24)부터는 Part 9로 넘어가 **Hugging Face**와 오픈소스 AI 모델 활용을 학습합니다. 이번 챕터에서 익힌 배포 지식은 Hugging Face Spaces 배포에도 그대로 적용됩니다.*